

Semester	S.E. Semester IV – CMPN
Subject	R programming
Subject Professor In-charge	Prof. Sanjeev D.
Assisting Teachers	Prof. Prakash p.
Laboratory	M414A

Student Name	Chetan katkar
Roll Number	23102A0024
Grade and Subject Teacher's Signature	

Experiment Number	03
Experiment Title	Multi-Source Retail Sales Data Integration and Analysis.
Git hub repo link	https://github.com/Chetan-Katkar/Air-Quality-Data-Cleaning-R/tree/main/Lab/R-lab-03
Objectives (Skill Set / Knowledge Tested / Imparted)	Objectives • To understand the process of importing heterogeneous data formats (CSV, JSON, and Excel) into R. • To perform data cleaning and preprocessing, including handling missing values and duplicates. • To integrate multiple data sources using dplyr join operations. • To conduct business analytics using group_by(), summarise(), and case_when(). • To establish SQLite database connectivity and perform SQL querying from R. • To apply critical reasoning to interpret results and extract meaningful business insights.

Description	A multinational retail company maintains its business data across multiple systems. Sales transactions are stored in CSV format, product information in JSON format, and customer information in Excel format. This practical involves developing an R-based solution to import, clean, integrate, and analyze these heterogeneous datasets to obtain a consolidated view of sales performance and customer value. Finally, the processed data is stored in an SQLite database for future analysis.
-------------	---

Code &
Output

```
# Install necessary packages if not present
if (!require("dplyr")) install.packages("dplyr")
if (!require("jsonlite")) install.packages("jsonlite")
if (!require("readxl")) install.packages("readxl")
if (!require("RSQLite")) install.packages("RSQLite")

library(dplyr)
library(jsonlite)
library(readxl)
library(RSQLite)

# 1. Import Datasets
transactions <- read.csv("transactions.csv", stringsAsFactors = FALSE)
products <- fromJSON("products.json")
customers <- read_excel("customers.xlsx")

# 2. Inspect and Clean Data

## Clean Transactions
# - Remove missing CustomerID or InvoiceNo
transactions_clean <- transactions %>%
  filter(!is.na(CustomerID), !is.na(InvoiceNo)) %>%
  distinct() %>%
  filter(Quantity > 0) # Invalid or zero quantities

## Clean Products
products_clean <- products %>%
  filter(!is.na(StockCode), !is.na(UnitPrice)) %>%
  distinct(StockCode, .keep_all = TRUE) %>%
  filter(UnitPrice > 0) # Invalid or zero unit prices

## Clean Customers
customers_clean <- customers %>%
  filter(!is.na(CustomerID)) %>%
  distinct(CustomerID, .keep_all = TRUE)

# 3. Create Revenue Attribute in Transactions
# Since UnitPrice is in products, we first need to join, or we wait for integration.
# The problem states 'Create a new attribute: Revenue = Quantity * UnitPrice' in Task 1.
# We will temporarily join UnitPrice to compute this, or do it during Integration (Task 2).
# We'll do it after integration to ensure we have UnitPrice available.

# Integrate data
# We use inner_join to ensure all transactions have matching product and customer information.
final_data <- transactions_clean %>%
  inner_join(products_clean, by = "StockCode") %>%
  inner_join(customers_clean, by = "CustomerID")

# Calculate Revenue
final_data <- final_data %>%
  mutate(Revenue = Quantity * UnitPrice)

# Verify dimensions
cat("Dimensions of final dataset:", dim(final_data), "\n")

# Identify unmatched records (using anti_join)
unmatched_products <- transactions_clean %>%
  anti_join(products_clean, by = "StockCode")
unmatched_customers <- transactions_clean %>%
  anti_join(customers_clean, by = "CustomerID")

cat("Unmatched transactions (missing products):", nrow(unmatched_products), "\n")
cat("Unmatched transactions (missing customers):", nrow(unmatched_customers), "\n")

... Dimensions of final dataset: 387877 9
Unmatched transactions (missing products): 4831
Unmatched transactions (missing customers): 0
```

```
# 1. Total sales revenue
total_revenue <- sum(final_data$Revenue)
cat("Total Sales Revenue:", total_revenue, "\n\n")

# 2. Top 5 products based on revenue
top_products <- final_data %>%
  group_by(StockCode, Description) %>%
  summarise(ProductRevenue = sum(Revenue), .groups = 'drop') %>%
  arrange(desc(ProductRevenue)) %>%
  head(5)
print("Top 5 Products by Revenue:")
print(top_products)
cat("\n")

# 3. Top 5 countries based on revenue
top_countries <- final_data %>%
  group_by(Country) %>%
  summarise(CountryRevenue = sum(Revenue), .groups = 'drop') %>%
  arrange(desc(CountryRevenue)) %>%
  head(5)
print("Top 5 Countries by Revenue:")
print(top_countries)
cat("\n")

# 4. Top 5 customers based on total purchase value
top_customers <- final_data %>%
  group_by(CustomerID) %>%
  summarise(TotalPurchase = sum(Revenue), .groups = 'drop') %>%
  arrange(desc(TotalPurchase)) %>%
  head(5)
print("Top 5 Customers by Purchase Value:")
print(top_customers)
cat("\n")

# Classify customers
customer_summary <- final_data %>%
  group_by(CustomerID, Country) %>%
  summarise(TotalPurchase = sum(Revenue), .groups = 'drop') %>%
  mutate(
    CustomerCategory = case_when(
      TotalPurchase < 1000 ~ "Low Value",
      TotalPurchase >= 1000 & TotalPurchase < 5000 ~ "Medium Value",
      TotalPurchase >= 5000 & TotalPurchase < 10000 ~ "High Value",
      TotalPurchase >= 10000 ~ "Premium"
    )
  )
print("Customer Classification Sample:")
print(head(customer_summary))

# Identify market performance
market_performance <- final_data %>%
  group_by(Country) %>%
  summarise(MarketRevenue = sum(Revenue), .groups = 'drop') %>%
  arrange(desc(MarketRevenue))

high_market <- head(market_performance, 1)
under_market <- tail(market_performance, 1)
cat("\nHigh-performing market:", high_market$Country, "with revenue:", high_market$MarketRevenue, "\n")
cat("\nUnderperforming market:", under_market$Country, "with revenue:", under_market$MarketRevenue, "\n")
```

```

*** Total Sales Revenue: 10752840

[1] "Top 5 Products by Revenue:"
# A tibble: 5 x 3
  StockCode Description                ProductRevenue
  <chr>      <chr>                      <dbl>
1 23843     PAPER CRAFT , LITTLE BIRDIE        168470.
2 47566     PARTY BUNTING                     142438.
3 22423     REGENCY CAKESTAND 3 TIER           135605.
4 85123A    WHITE HANGING HEART T-LIGHT HOLDER  93746.
5 23166     MEDIUM CERAMIC TOP STORAGE JAR     81033.

[1] "Top 5 Countries by Revenue:"
# A tibble: 5 x 2
  Country      CountryRevenue
  <chr>        <dbl>
1 United Kingdom 8861857.
2 Netherlands   363884.
3 EIRE           331660.
4 Germany       263819.
5 France        226976.

[1] "Top 5 Customers by Purchase Value:"
# A tibble: 5 x 2
  CustomerID TotalPurchase
  <dbl>      <dbl>
1 18102      488760.
2 14646      357531.
3 17450      186038.
4 14911      182691.
5 16446      168472.

[1] "Customer Classification Sample:"
# A tibble: 6 x 4
  CustomerID Country      TotalPurchase CustomerCategory
  <dbl> <chr>      <dbl> <chr>
1 12346 United Kingdom 77184. Premium
2 12347 Iceland       5438. High Value
3 12348 Finland       1790. Medium Value
4 12349 Italy          1927. Medium Value
5 12350 Norway         488. Low Value
6 12352 Norway        1617. Medium Value

High-performing market: United Kingdom with revenue: 8861857
Underperforming market: Saudi Arabia with revenue: 181.44

```

```
# Create SQLite database and export final dataset
con <- dbConnect(RSQLite::SQLite(), "retail_sales.sqlite")
dbWriteTable(con, "retail_sales", final_data, overwrite = TRUE)

# Query 1: Top 5 customers based on revenue
query1 <- "
SELECT CustomerID, SUM(Revenue) as TotalRevenue
FROM retail_sales
GROUP BY CustomerID
ORDER BY TotalRevenue DESC
LIMIT 5
"
cat("Top 5 Customers (SQL Query):\n")
print(dbGetQuery(con, query1))

# Query 2: Total revenue by country
query2 <- "
SELECT Country, SUM(Revenue) as TotalRevenue
FROM retail_sales
GROUP BY Country
ORDER BY TotalRevenue DESC
"
cat("\nTotal Revenue by Country (SQL Query - Top 5 shown):\n")
print(head(dbGetQuery(con, query2), 5))

# Disconnect from database
dbDisconnect(con)
```

Top 5 Customers (SQL Query):

	CustomerID	TotalRevenue
1	18102	408760.0
2	14646	357531.1
3	17450	186038.0
4	14911	182690.5
5	16446	168472.5

Total Revenue by Country (SQL Query - Top 5 shown):

	Country	TotalRevenue
1	United Kingdom	8861857.1
2	Netherlands	363884.5
3	EIRE	331660.2
4	Germany	263819.0
5	France	226975.6

Conclusion

The practical helped in understanding the end-to-end data integration workflow in R. Various operations were successfully performed to import, clean, and combine multi-source datasets (CSV, JSON, Excel) using the dplyr package. The practical provided hands-on experience in business analytics, classifying customers, and identifying top-performing products and markets. Furthermore, it demonstrated the integration of R with SQLite for persistent data storage, ultimately enabling data-driven business insights.